

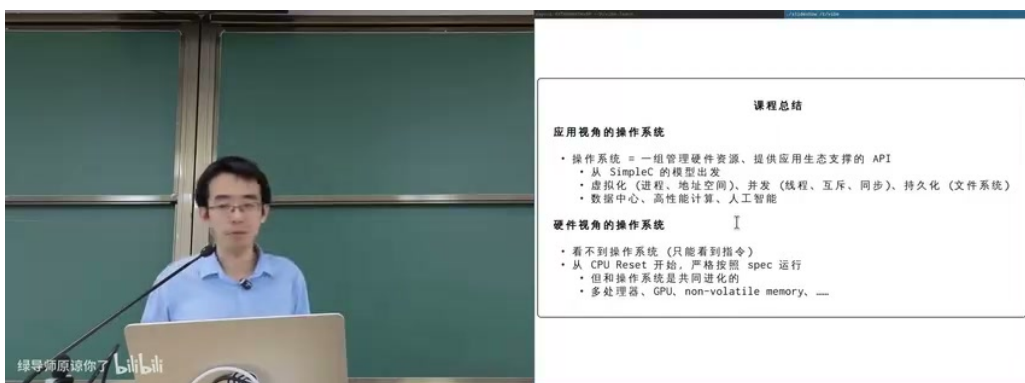
课程笔记

30 - Agentic AI 和我们的未来；课程总结

2026 南京大学操作系统原理

lecture-to-notes

2026-06-20



视频作者/频道：南京大学操作系统原理，蒋炎岩

发布日期：本地课程视频

视频时长：01:24:25

视频链接：未填写，本笔记基于本地视频文件与清洗转录稿生成

目录

1 总论：操作系统到底是什么	3
1.1 应用程序的需求是第一性原理	3
1.2 硬件视角：CPU 看不见操作系统	4
1.3 Agentic AI 操作系统不是从裸机重来	4
1.4 本章小结	5
2 复习方法：Unix Philosophy 与 AI 时代的覆盖率	5
2.1 考试不是死记硬背，而是重走推理链	5
2.2 Unix 与 Windows：两种生态哲学	5
2.3 Content as code：把课程材料变成可查询对象	6
2.4 AI 辅助复习：从押题到覆盖率	7
2.5 本章小结	8
3 课程技术主线：从程序到系统	8
3.1 序论：程序、状态机与系统调用边界	8
3.2 Virtualization：世界如何被构建出来	9
3.3 Concurrency：打开多处理器的盒子	10
3.4 Persistence：文件系统是块设备上的数据结构课	10
3.5 Systems 的共同模式	11
3.6 本章小结	11
4 从学习操作系统到理解时代变化	12
4.1 Teaching is a powerful tool to learn	12
4.2 Scaling law：世界变化不是偶然	12
4.3 Pre-training、post-training、test-time scaling	13
4.4 本章小结	14
5 VibeOS：一个关于未来界面的寓言	14
5.1 从“AI 写代码”到“没有代码的界面”	14
5.2 为什么它仍然是操作系统问题	15
5.3 时代的结束，未来的开始	15
5.4 本章小结	16
6 总结与延伸	16
6.1 这门课真正留下什么	16
6.2 在时代里找到自己的位置	16
6.3 面向未来的三个练习	17
6.4 拓展阅读	17

6.5 本章小结	18
--------------------	----

1 总论：操作系统到底是什么

本讲是 2026 年南京大学操作系统原理的最后一讲。它不是一节普通的复习课：前半段把整门课从“应用程序想要什么”的角度重新压缩，后半段把这个视角延伸到 Agentic AI、未来的交互界面和学习方式。课堂开始时，老师再次回到第一讲反复出现的定义：

操作系统的应用视角定义

从应用程序的角度看，操作系统是一组管理硬件资源、提供应用生态支撑的 API。更具体地说，它是一组系统调用，以及围绕这些系统调用形成的运行时、工具链、文件系统、进程模型和应用生态。

这个定义在第一讲时可能只是口号；学完整门课后，它已经可以被拆成一组可以推理的事实：

- 应用程序在一个被虚拟化的执行环境中运行，这个环境就是进程。
- 进程拥有地址空间、文件描述符、线程、信号、终端、环境变量等资源。
- 当进程需要改变操作系统状态、访问外部世界或持久化信息时，就必须通过系统调用越过用户态边界。
- 操作系统把硬件能力包装成可组合、可控制、可隔离的抽象，使应用程序不必直接面对复杂硬件。

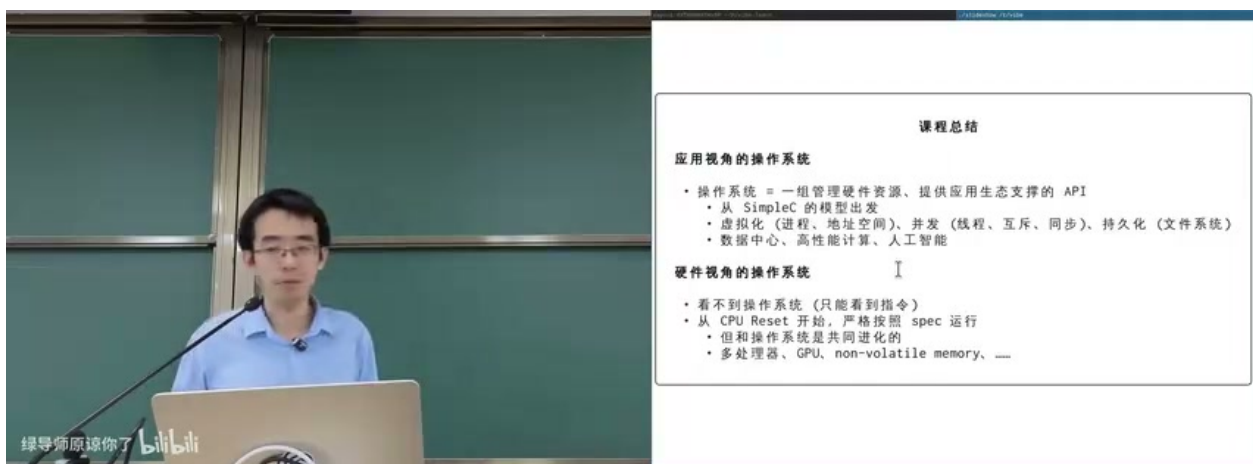


图 1: 课程总结的第一张实质性幻灯片：从应用视角和硬件视角同时定义操作系统。¹

1.1 应用程序的需求是第一性原理

老师在这一讲中反复强调，操作系统的机制不是凭空出现的。正确的学习方式不是背诵零散事实，而是从应用需求出发追问：

¹视频画面时间区间：00:00:30–00:00:35。

1. 应用程序想做什么？
2. 这件事只靠 CPU 和内存中的纯计算能否完成？
3. 如果不能，操作系统需要暴露什么对象和操作？
4. 暴露之后，是否会产生隔离、安全、一致性或性能问题？

例如，一个程序想“创建另一个程序”，于是需要进程管理 API；想在原本非法的地址范围放入内存，于是需要 `mmap` 之类的地址空间 API；想让多个执行流共享数据，于是需要线程、互斥、同步；想留下持久化痕迹，于是需要文件系统、块设备和崩溃一致性。

没有魔法，只有状态和边界

计算机系统里没有魔法。再奇怪的行为，也必然来自某个状态被复制、共享、缓存、延迟写入、并发交错或跨越边界。学操作系统的价值之一，就是把“看起来像魔法”的行为还原为状态机与接口设计。

1.2 硬件视角：CPU 看不见操作系统

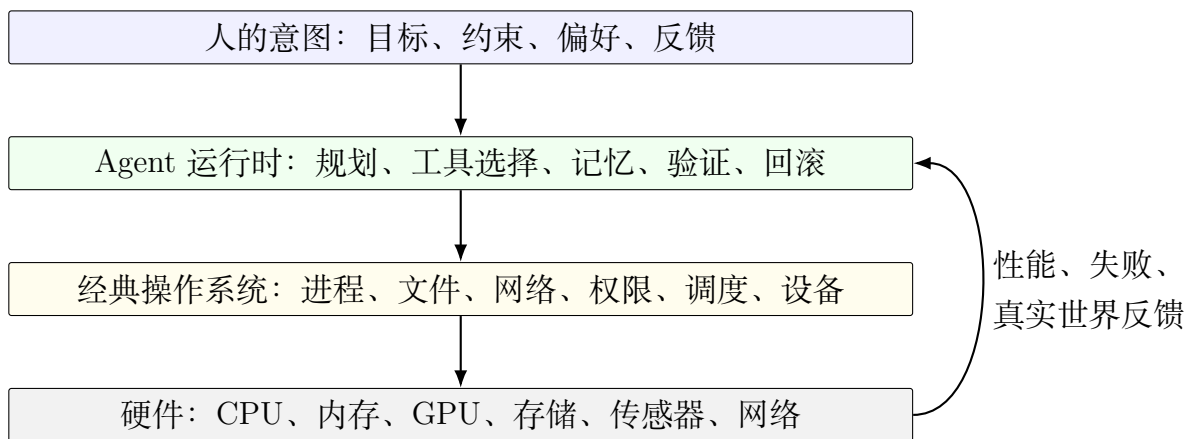
另一个视角是硬件。CPU 本身并不知道“操作系统”的概念，它只做一件事：按照手册执行合法、有权限的指令序列。固件、内核、用户程序最终都变成指令；只要特权级和指令语义允许，CPU 就照单执行。

但是 CPU 与操作系统又在共同进化。单处理器时代的内核、面向多核的同步原语、GPU 与加速器驱动、non-volatile memory 的持久化语义，都说明硬件变化会迫使操作系统重新定义抽象。操作系统不是固定教材条目，而是一套随应用需求和硬件能力一起演化的系统软件。

1.3 Agentic AI 操作系统不是从裸机重来

本讲的标题包含 Agentic AI。老师提出一个谨慎但重要的判断：如果未来需要“agent 的操作系统”，它大概率不是从裸机重新发明一套传统内核，而是像 Android/iOS 借助 Linux 一样，在经典操作系统之上叠加新的运行时、权限模型、工具接口、任务调度和交互层。

这和编程语言运行时的演化相似。早期 Unix 应用常常是 C 程序编译成二进制后直接在内核 API 上运行；今天的应用则常常写在 Python、JavaScript、Swift、WebAssembly、JVM 或浏览器运行时之上。Agentic AI 时代的“操作系统”也可能是类似的上层封装：底层依然是 Linux、文件、进程、网络、GPU、沙箱和权限；上层变化的是意图表达、工具调用、长程任务和世界模型。



1.4 本章小结

操作系统的核心不是“内核很大”，而是“应用程序通过哪些稳定边界获得硬件能力”。这一定义足以覆盖从单片机、手机、服务器到 AI 推理集群的系统形态，也为理解 Agentic AI 时代的新运行时提供了起点：新的抽象会出现，但旧的资源、边界和状态不会消失。

2 复习方法：Unix Philosophy 与 AI 时代的覆盖率

2.1 考试不是死记硬背，而是重走推理链

老师对考试的建议很明确：如果已经理解课堂上的代码 demo、基本原理、涉及的系统调用和概念，就不需要过度焦虑。考试会考具体知识点，但更重要的是能否从 first principle 推理出合理答案。

这背后有一个训练目标：短期记忆会忘，但“操作系统还可以这么做”的印象会留下。当你以后遇到前端、后端、AI infra、数据库、容器、编译器或安全问题时，可能会突然意识到：自己学过进程地址空间、文件描述符、动态链接、崩溃一致性和并发 interleaving，这些概念仍然能解释新系统。

AI 帮你写作业时，谁在被训练

如果老师用 AI 出题，学生用 AI 写作业，唯一可能没有得到训练的人就是学生。AI 时代更需要主动设计自己的训练过程：让模型辅助拆解、生成练习、检查漏洞，而不是把理解外包出去。

2.2 Unix 与 Windows：两种生态哲学

本讲用 Unix philosophy 作为复习的入口。Windows 和 Unix 都提供 API，但应用生态哲学不同：

- Windows 的代表性体验偏向大型单体应用：Office、Visual Studio、Adobe 工具链。一个应用尽可能解决一个大任务，降低新手入口成本。

- Unix 的代表性体验偏向小工具组合：每个工具做好一件事，通过文件、管道、标准输入输出、进程和脚本互相协作。

Unix 对新手不友好，因为命令行把整个开放世界摆在你面前：工具太多、概念太多、反馈太硬。但一旦掌握组合，你获得的是系统级的可编程性。老师把这件事和现代 Coding Agent 联系起来：即使未来很多代码对普通用户不可见，计算机专业学生仍要理解“可组合工具”的力量。

Unix philosophy 在 AI 时代没有过时

当大语言模型成为管道的终点，Unix philosophy 反而更强：find、sort、grep、去重、复制到剪贴板、再交给 AI 整理，仍然是一条清晰的数据流。AI 不是替代组合思想，而是成为组合链条中的一个更强算子。

2.3 Content as code: 把课程材料变成可查询对象

老师展示了一个很系统的复习动作：因为课件已经是 content as code，所以可以用命令行收集所有展示过的 demo。流程大致是：

1. 用 find 找到课程目录中的源文件。
2. 用 sort -V 按 lecture 编号自然排序。
3. 用 grep 筛选真正的 OS demo，排除 review/comments 等干扰项。
4. 去重并复制到剪贴板。
5. 把得到的 demo 列表交给 AI 或自己整理成复习提纲。

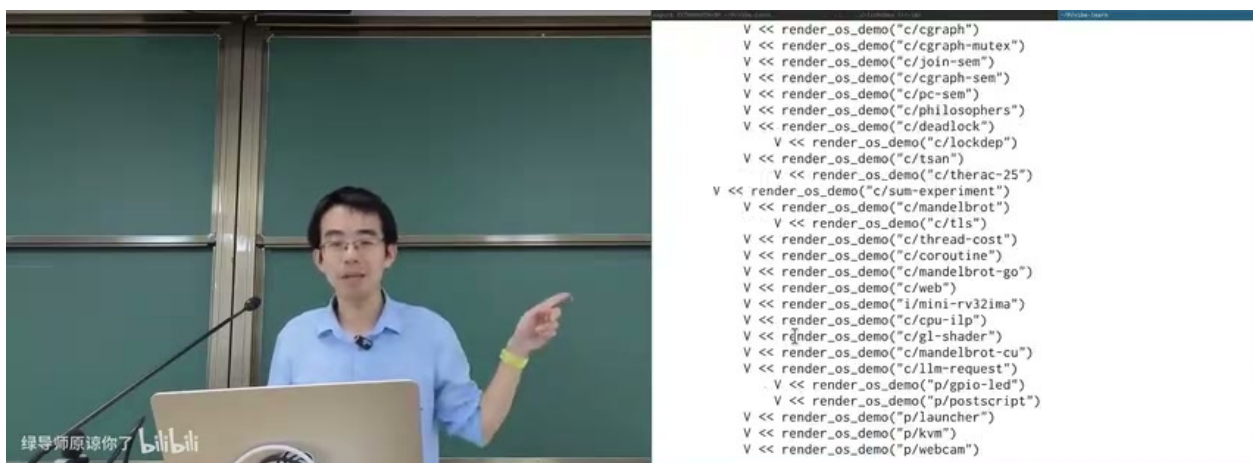


图 2：命令行组合得到本学期展示过的 demo 列表：复习材料首先被转化为可处理的数据流。²

²视频画面时间区间：00:18:40-00:18:45。


```

1 find os2026 -name "*.py" \
2   | sort -V \
3   | grep "render_os_demo" \
4   | uniq

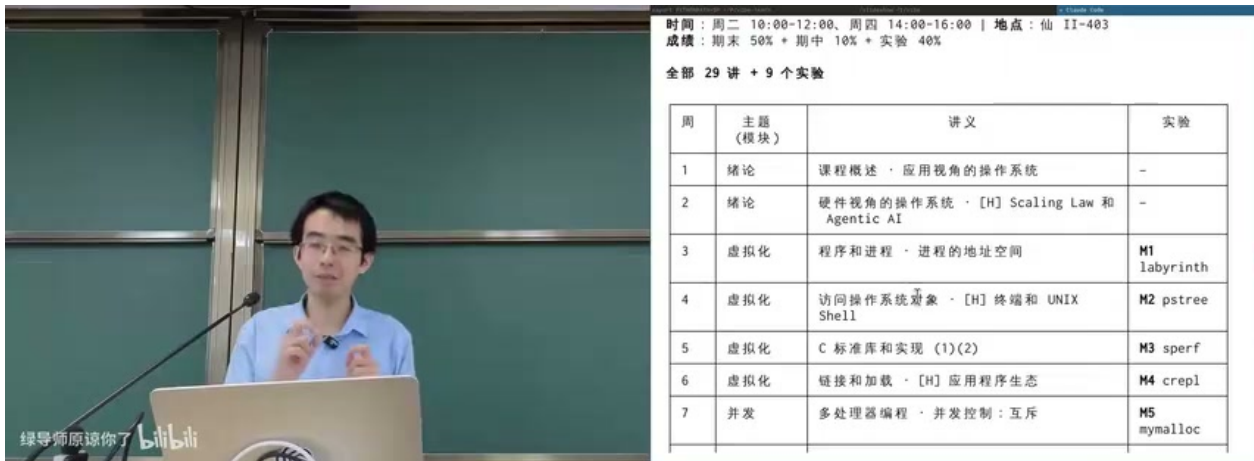
```

Listing 1: 课堂演示背后的 Unix 风格数据流示意

这段流程的重点不是要背命令，而是看到“工具之间可以合作”：文件系统、标准输出、排序、过滤、去重、剪贴板和 AI 都能接成一条链。这个链条就是操作系统提供的生态。

2.4 AI 辅助复习：从押题到覆盖率

老师现场让 AI 生成期末题，并把结果当作讨论材料。AI 生成的题目并不代表真正考试，但它暴露出一种有效复习方式：不要只让 AI 根据全部材料猜一套卷子，而要让它围绕每个知识点生成多组题目，从而提高覆盖率。



时间：周二 10:00-12:00、周四 14:00-16:00 | 地点：仙 II-403
成绩：期末 50% + 期中 10% + 实验 40%

全部 29 讲 + 9 个实验

周	主题 (模块)	讲义	实验
1	绪论	课程概述 · 应用视角的操作系统	-
2	绪论	硬件视角的操作系统 · [H] Scaling Law 和 Agentic AI	-
3	虚拟化	程序和进程 · 进程的地址空间	M1 labyrinth
4	虚拟化	访问操作系统对象 · [H] 终端和 UNIX Shell	M2 pstree
5	虚拟化	C 标准库和实现 (1)(2)	M3 sperf
6	虚拟化	链接和加载 · [H] 应用程序生态	M4 crepl
7	并发	多处理器编程 · 并发控制：互斥	M5 mymalloc

图 3: 课程讲义与实验列表被整理成复习索引：每一讲都可以变成知识点与练习生成的入口。³

这和高考复习中的“五三”类似：模拟卷是验收，真正的复习是对一个范围内的题型和概念做足够穷尽的覆盖。AI 的价值在于用低成本生成大量变体，再让学生检查、归纳、反推知识点。

³视频画面时间区间：00:20:45-00:20:50。

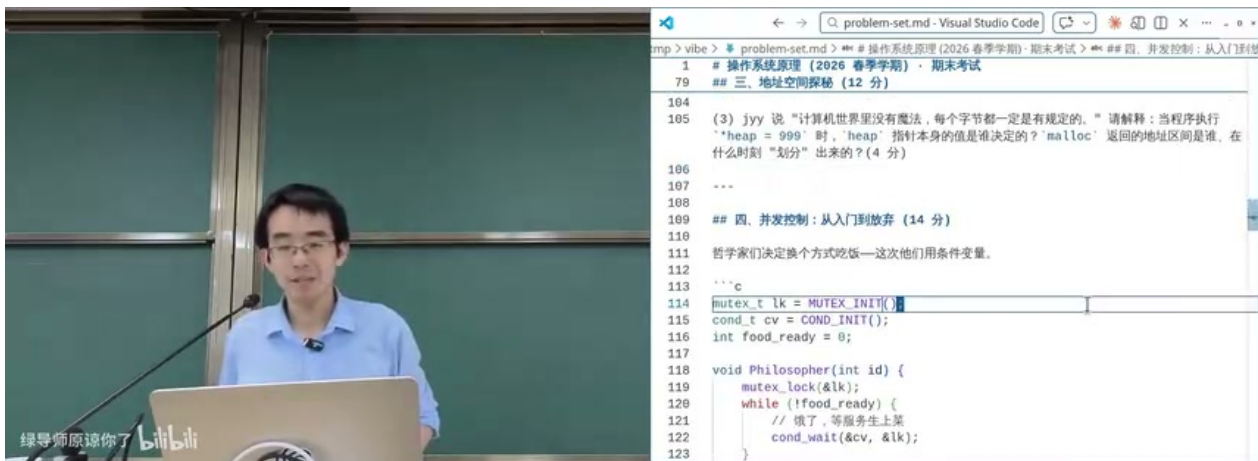


图 4: AI 生成的并发题片段：老师借此强调条件变量万能同步方法和 interleaving 检查。⁴

并发题是一个典型例子。老师提醒，条件变量的万能同步方法几乎是送分项；如果选择更聪明的信号量或无锁方案，就要承担被 interleaving 找出错误的风险。AI 可以帮你生成题，但真正的训练是自己像模型检查器一样遍历状态空间。

复习操作系统的三层方法

1. **代码层**：把课堂 demo 编译运行，观察真实行为。
2. **机制层**：解释 demo 背后的系统调用、状态复制、共享对象或一致性约束。
3. **设计层**：问如果由自己设计这个 API，会如何满足应用需求、避免错误、暴露能力。

2.5 本章小结

复习不是把所有 lecture notes 一次性塞给 AI，然后等待押题结果。更好的方式是把课程材料结构化，围绕每个知识点生成练习，自己检查推理链和状态空间。Unix philosophy 提供了“把世界接起来”的工具观，AI 则把这种组合能力推到新的尺度。

3 课程技术主线：从程序到系统

3.1 序论：程序、状态机与系统调用边界

课程的起点不是内核源码，而是已有知识：数字逻辑电路、RISC-V 处理器模拟器、程序执行模型、SimpleC 和汉诺塔。老师用这些例子把“程序是什么”展开为状态机：

- 逻辑门可以用代码连接，电路可以模拟。
- CPU state 包含寄存器、CSR、内存和 PC，指令不断更新状态。

⁴视频画面时间区间：00:26:40–00:26:45。

- 递归程序可以展开成显式栈帧，每个栈帧都要记住下一步执行位置。
- 编译器优化必须保持顺序执行语义，但系统调用是优化不可穿过的边界。

系统调用之所以特殊，是因为执行系统调用时，控制流进入内核；回来之后，进程看到的世界可能已经被改变。内存映射可以变，文件描述符状态可以变，信号可以到来，其他进程可以运行。系统调用把程序从纯计算带入共享世界。

系统调用是外部世界的入口

在用户态内部，程序主要改变自己的寄存器和内存；一旦执行系统调用，它就在请求操作系统改变某个全局状态：进程树、地址空间、文件偏移、设备状态、持久化存储或调度关系。

3.2 Virtualization: 世界如何被构建出来

虚拟化部分回答“进程看到的世界从哪里来”。从 CPU reset 到 firmware，再到内核加载、第一个 init 进程、之后所有进程通过系统调用驱动操作系统状态变化，整个用户世界就被逐步构建出来。



图 5: 课程地图的前半部分：从 Introduction 进入 Virtualization，覆盖进程、地址空间、文件描述符与动态链接。⁵

这一部分的核心概念包括：

⁵视频画面时间区间：00:38:25–00:38:30。

主题	应用需求与系统机制
进程树	应用需要创建、等待、终止、替换程序；操作系统提供 <code>fork/execve/wait</code> ，并维护父子关系。
地址空间	应用需要私有内存和按需分配；操作系统提供虚拟内存、 <code>mmap</code> 、页表、缺页异常和进程间内存访问边界。
文件描述符	应用需要持有操作系统对象；Unix 用小整数引用文件、管道、终端、socket、设备等对象。
动态链接	应用需要复用库并在运行时装配；符号解析、PLT/GOT 和预加载机制也带来 hook 与 interposition 的可玩性。

老师特别强调文件描述符和 Unix 设计哲学之间的关系。父子进程共享文件描述符，并不是偶然的历史包袱，而是为了让进程之间通过继承、重定向和管道自然合作。一个程序可以不必理解另一个程序的内部，只要双方遵守文件接口，就能组合成更大的系统。

3.3 Concurrency: 打开多处理器的盒子

并发部分从一个进程内部的多个执行流展开。共享内存让线程之间通信极快，几条指令的延迟就能传递数据；但它同时带来 data race、atomicity violation、deadlock、memory model 和 interleaving 爆炸。

老师把操作系统称为最早的并发程序之一：即使每个用户进程单线程，多个进程通过系统调用进入内核，内核代码也会在中断、抢占、多核和共享内核地址空间中并发执行。因此操作系统研究天然产生了互斥、同步、死锁检测、数据竞争检测和模型检查等问题。

聪明同步的风险

如果题目可以用条件变量万能同步方法稳定解决，考试中选择更短、更巧的同步方案并不总是划算。并发程序的正确性不是看起来合理，而是所有关键 interleaving 都不出错。

3.4 Persistence: 文件系统是块设备上的数据结构课

持久化部分从设备驱动开始：LED、核弹发射器式的设备模型、磁盘、Webcam、NVMe SSD。设备通常有寄存器、状态位和数据端口；操作系统通过驱动把它们接入统一接口。在 Linux 中，`struct file_operations` 是把设备变成“文件”的关键桥梁。

文件系统则是在块设备上实现的数据结构。块设备按块读写，文件系统要把“文件和目录”的 API 映射到块分配、索引、目录项、权限、缓存和崩溃恢复。课堂中多次出现思想包括：

- append-only 结构可以把随机写转化为顺序追加，再用索引支持随机读。
- crash consistency 要考虑任意时刻断电后磁盘处于哪个前缀状态。

- OverlayFS、FUSE、GalgameFS 等例子说明文件系统也可以虚拟出来，只要满足文件/目录 API。

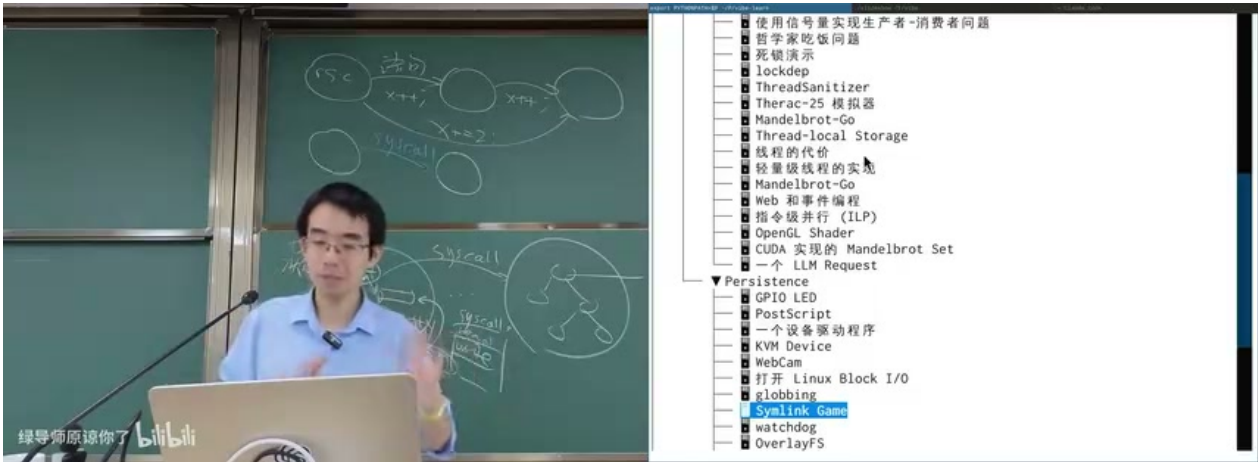


图 6: 课程地图的后半部分：并发、持久化、安全、虚拟机与容器共同体现“抽象复杂硬件，暴露可用能力”。⁶

3.5 Systems 的共同模式

回看整门课，反复出现的是同一个模式：底层很复杂，上层需要干净模型，但抽象不能把底层能力完全藏死。

复杂底层	操作系统抽象	暴露给应用的力量
CPU 特权级、异常、中断	进程、系统调用	创建程序、隔离错误、共享机器
MMU、页表、物理内存	地址空间、mmap	私有内存、按需分配、共享映射
多核、缓存一致性	线程、锁、条件变量	并行计算、低延迟共享数据
块设备、掉电风险	文件系统、日志、copy-on-write	持久化名字空间、崩溃恢复
设备寄存器、DMA、中断	文件接口、驱动	统一 I/O、可脚本化设备控制

这也是“从需求出发解构问题”的意义。操作系统不是自上而下发明漂亮抽象，而是回应应用程序不断提出的新需求：更快、更安全、更可组合、更少出错、更容易恢复。

3.6 本章小结

课程技术主线可以压缩为三句话：程序是状态机；系统调用让程序接触共享世界；操作系统用进程、地址空间、文件描述符、并发模型和文件系统把复杂硬件组织成可用生态。

⁶视频画面时间区间：00:51:25-00:51:30。

掌握这些 key ideas 后，很多细节可以重新推导，而不是死记硬背。

4 从学习操作系统到理解时代变化

4.1 Teaching is a powerful tool to learn

老师在非技术回顾中谈到，教操作系统本身也是一种学习工具。每一年重新讲课，都可能发现旧讲法不够好、某个概念可以换一种更清楚的展开、某个研究想法在课堂例子中提前露出影子。

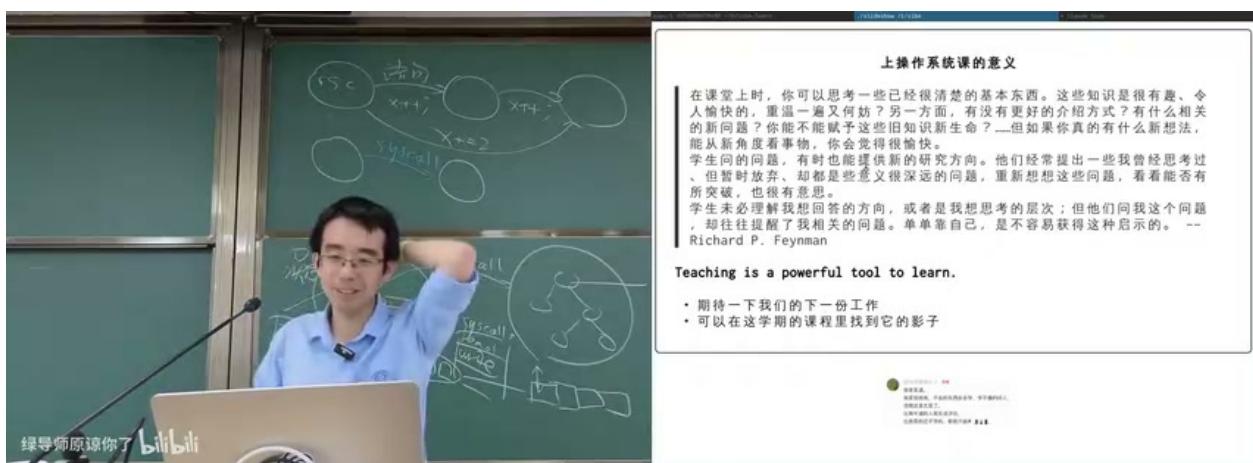


图 7：“上操作系统课的意义”：教学既是课程输出，也是教师持续学习和重新组织知识的方式。⁷

对学生而言，对应的方法是“重新发明一次”。例如问“什么是 Git”时，不应先搜索零散命令，而应从需求推导：多人协作、本地开发、版本同步、多版本管理、回退、分支。这样就能推导出 Git 是管理文件系统快照的数据结构，提交形成有向无环图，branch、stash、worktree 都是这个数据结构上的自然操作。

理解的标志

真正理解一个机制，不是能背出它的所有选项，而是忘掉细节后，仍能从应用需求和约束重新设计出相近的方案。若能把自己的设计和真实系统对照修正，就完成了一次有效学习。

4.2 Scaling law：世界变化不是偶然

课堂从国际象棋、存储设备、Intel、NVIDIA 和大语言模型谈 scaling law。老师的核心判断是：当某个能力沿指数曲线增长时，一个领域的结构性变化就会出现。深蓝击败世界冠军、AlphaGo 改写围棋、Google 建立在可扩展存储与搜索之上、GPU 从图形走向 AI，都不是偶然的孤立事件。

⁷ 视频画面时间区间：00:55:20–00:55:25。

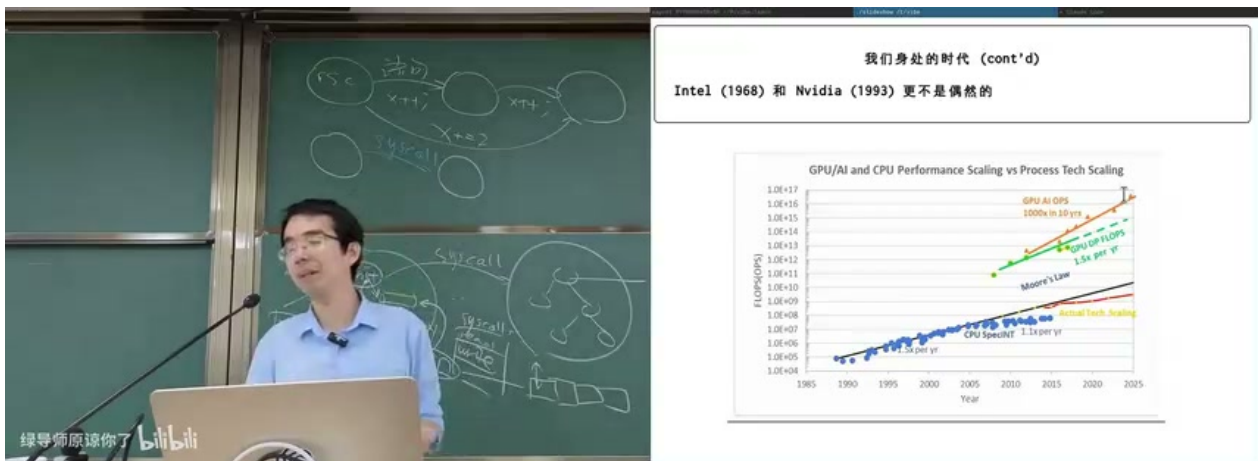


图 8: GPU/AI 与 CPU 性能扩展曲线：对数坐标上的直线意味着指数级增长。⁸

对数坐标上的直线容易让人麻木，但它代表指数增长。指数增长的危险在于，早期看起来只是“玩具变好一点”，越过某个阈值后就会重塑整个生态。操作系统课中的 first principle 训练，正是为了在变化的萌芽阶段看出结构趋势。

4.3 Pre-training、post-training、test-time scaling

老师把 AGI 的逼近概括为三个 scaling 维度：

- **pre-training scaling**: 更多数据、更好数据、更大模型、更长训练带来基础能力增长。
- **post-training scaling**: 在有明确反馈信号的任务上，通过监督、偏好、强化学习或自我改进提高专门能力。
- **test-time scaling**: 推理时投入更多计算，把任务拆解、搜索、验证、反思、调用工具，提升单次解题能力。



图 9: AGI 的逼近：pre-training、post-training 与 test-time scaling 共同提高模型能力上限。

9

⁸视频画面时间区间：00:58:55-00:59:00。

课堂中用复习考试举例说明 test-time scaling：不是让 AI 一次性猜卷，而是让它先列出 100 个知识点，再为每个知识点生成题目、答案、变体和检查脚本。只要算力足够，测试时搜索空间可以被系统性展开。这对学习者既是机会，也是压力。

AGI 不是“更会聊天”而已

如果 test-time scaling 可以把推理、搜索、工具调用和自我验证接起来，那么智能系统的能力上限不只取决于模型参数，也取决于运行时 harness。未来的竞争很可能是模型、工具、验证环境、数据反馈和任务组织方式的共同竞争。

4.4 本章小结

操作系统课训练的是一种看规律的方式：从需求出发、识别边界、观察 scaling、推导机制。AI 时代不是让这些能力失效，而是把问题尺度放大。学生需要学会用 AI 生成材料、检查盲区、设计训练，同时保留自己的推理链。

5 VibeOS：一个关于未来界面的寓言

5.1 从“AI 写代码”到“没有代码的界面”

最后一段课堂播放了 VibeOS 演示。演示者称它为“世界上第一个完全幻觉出来的操作系统”：可以启动，有 Notepad、Calculator、Internet Explorer 等窗口，但这些应用没有传统意义上的按钮、事件处理器或业务逻辑，界面和行为由 AI 实时生成。

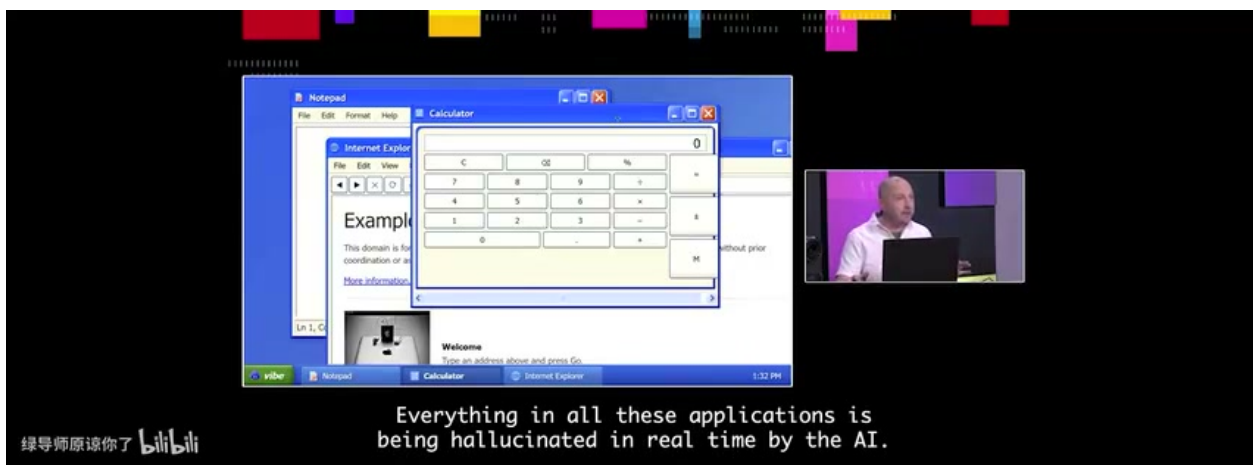


图 10: VibeOS 演示中的 Notepad、Calculator 和 Internet Explorer：应用界面被描述为由 AI 实时幻觉生成。¹⁰

这个例子当然带有戏谑和夸张，但它击中了一个真实趋势：传统软件把行为固化在代码里，用户通过按钮、菜单和表单触发预先写好的逻辑；生成式界面则可能把用户意图、当

⁹视频画面时间区间：01:06:30–01:06:35。

¹⁰视频画面时间区间：01:15:10–01:15:15。

前上下文和模型世界知识结合起来，临时生成“看起来像应用”的交互。

5.2 为什么它仍然是操作系统问题

VibeOS 看起来像在消解操作系统：如果 UI、内容和应用逻辑都可以由模型临时生成，是否还需要 API、文件、进程、权限、持久化？恰恰相反，它提出了新的操作系统问题：

- **权限**：幻觉出来的应用能访问哪些文件、网络、摄像头、账号和支付接口？
- **可验证性**：如果界面实时生成，用户如何知道动作真实发生、结果可复现？
- **持久化**：临时生成的应用状态如何保存、迁移、审计和回滚？
- **资源管理**：长程 agent、模型推理、工具调用和 GPU 资源如何调度？
- **隔离**：模型生成的操作如何被沙箱化，避免越权和连锁失败？

世界模型之下仍有系统边界

VibeOS 的意义不在于“从此不需要代码”，而在于用户界面可能从固定程序变成世界模型的投影。只要投影要影响真实世界，就仍然需要操作系统式的资源、权限、隔离、一致性和审计。

5.3 时代的结束，未来的开始

播放完演示后，老师回到自己的 slide：“时代的结束，未来的开始”。他把自己的课程、即将开设的 Generative Software Engineering、课程内容 public/open/source 化的愿景，以及学生未来的位置联系起来。

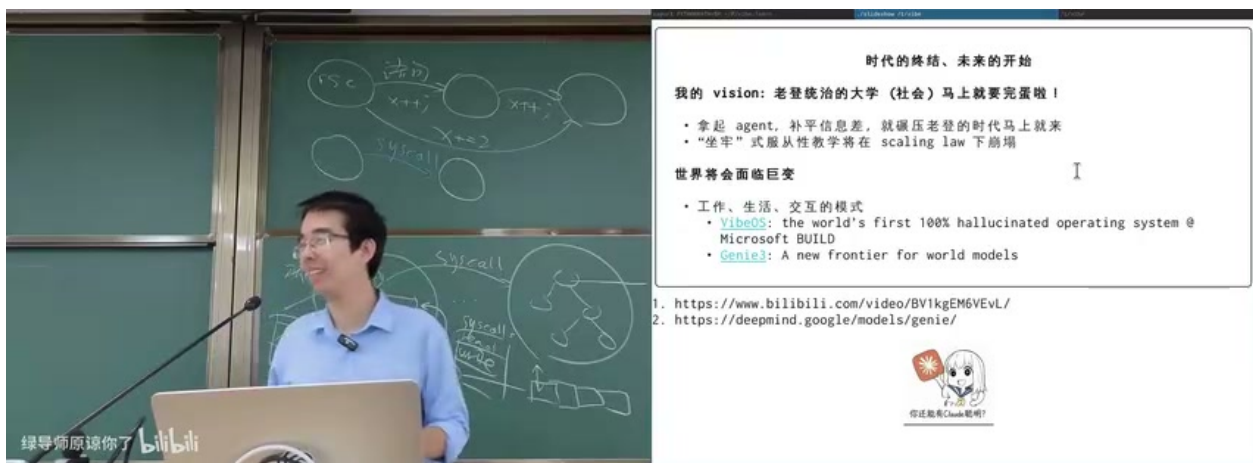


图 11：未来操作系统的思考：世界模型、实时生成场景和 VibeOS 类演示共同指向交互方式变化。¹¹

¹¹ 视频画面时间区间：01:20:35–01:20:40。

这里的“结束”不是说操作系统知识无用，而是说一种课程形态、软件形态和学习形态正在结束。过去，学生可能为了考试、保研、就业而学习；未来，这些“船票”仍然存在，但不再保证安全。模型能力、算力、工具、自动化和交互方式都在 scale，社会规则会被重新洗牌。

5.4 本章小结

VibeOS 是寓言也是技术信号。它把“软件是什么”从静态代码推向实时生成的交互，把“操作系统是什么”从管理硬件推向管理 agent、模型、工具、权限与真实世界反馈。经典操作系统不会消失，但它会成为新抽象的地基。

6 总结与延伸

6.1 这门课真正留下什么

如果把整门课压缩到最小集合，可以留下以下几个核心命题：

1. 程序是状态机；操作系统管理多个状态机与外部世界的交互。
2. 系统调用是用户程序与共享世界之间的边界。
3. 进程、地址空间、文件描述符、线程、文件系统都是对复杂硬件和复杂需求的抽象。
4. 好抽象降低心智负担，但不能抹掉底层复杂性；因此必须理解抽象泄漏的时刻。
5. Unix philosophy 的组合思想在 AI 时代更重要，因为 AI 可以成为工具链的一环。
6. 学习的目标不是记住所有 fact，而是能从需求和约束重新推导机制。

6.2 在时代里找到自己的位置

本讲最后的幻灯片不是技术清单，而是对学生的直接建议：在时代里找到自己的位置。

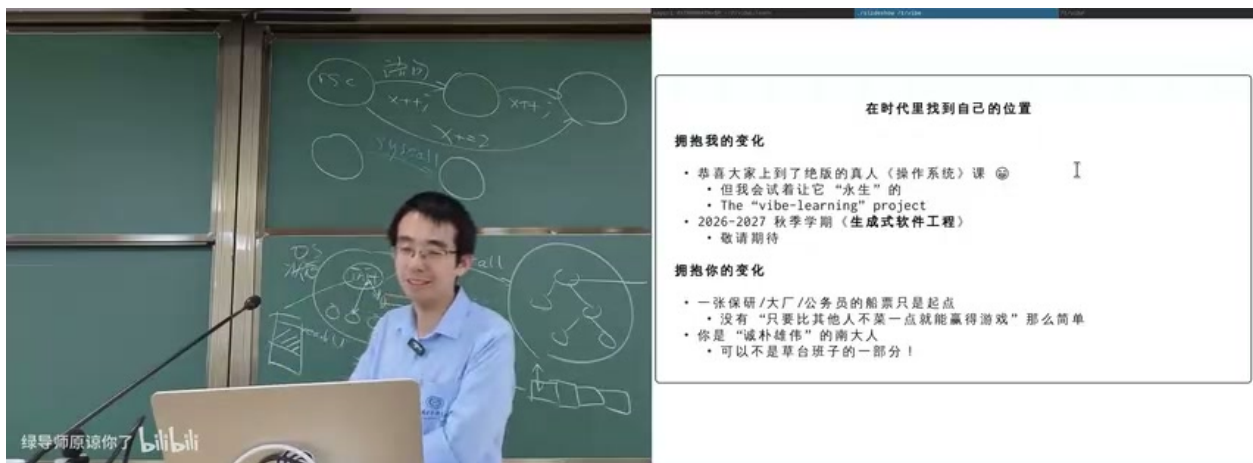


图 12: 课程最后的落点：时代变化会改写船票的意义，每个人都要重新寻找自己的位置。¹²

老师承认学生来听复习课，常常也是为了考试、保研、就业这些现实目标。但他提醒，这些目标只是起点。过去的游戏似乎是“只要比别人不菜一点就能赢”；AI scaling 之后，社会把人直接放到更大的竞争和协作场中。真正重要的问题变成：

- 你能否用 agent 补足信息差，而不是被工具替代？
- 你能否设计自己的学习 harness，让 AI 帮你变强而不是替你逃避训练？
- 你能否把一个领域的问题重新表述为需求、状态、接口、反馈和验证？
- 你是否愿意追求一个未必热门、但真正属于自己的方向？

最后的课程精神

知识有边界，梦想可以没有边界。操作系统课给出的不是一张万能船票，而是一套面对复杂系统的思考方式：把世界拆成状态、接口、资源、反馈和约束，然后主动构造能让自己前进的工具链。

6.3 面向未来的三个练习

为了把本讲内容转化为行动，可以做三个练习：

1. **重跑 demo**：选择 5 个课堂 demo，从源码、编译、运行、系统调用行为到背后的 first principle 完整解释一遍。
2. **重设计机制**：任选一个机制，如文件描述符继承、mmap、条件变量、append-only 文件系统，从应用需求重新设计，再与真实系统对照。
3. **构造 AI 复习 harness**：让 AI 列出知识点、生成题目、给出标准解、设计反例、检查自己的答案；但每一步都保留自己的判断和修正。

6.4 拓展阅读

- Unix philosophy：理解小工具、管道、文本接口和组合生态。
- OSTEP：把 virtualization、concurrency、persistence 三条主线与教材叙述互相校准。
- Linux man pages：把课堂概念落实到 fork、execve、mmap、open、pipe、fsync 等具体 API。
- 现代 AI coding agent 文档与实践：重点观察工具调用、沙箱、权限、回滚、测试和验证，而不仅仅是 prompt。

¹²视频画面时间区间：01:21:15–01:21:20。

6.5 本章小结

这节课的终点不是“课程结束”，而是把操作系统的思考方式带到新世界。未来的界面可能由模型生成，未来的课程可能持续 open/source 化，未来的工作可能由 agent 协作完成；但资源、边界、状态、一致性、反馈和抽象仍然是系统问题。学过操作系统的人，至少拥有了一套理解这些变化的语言。